

LLM-based Compatibility Detection

Abstract—xxxxxxx[?]

I. INTRODUCTION

II. BACKGROUND

A. API Compatibility Detection

The research necessity of API compatibility detection is driven by the combination of practical pain points of developers, technical evolution challenges, industry value demands, and gaps in existing solutions. In open-source communities such as GitHub, a large number of developer complaints indicate that version changes of third-party dependency package APIs often cause project errors—for example, runtime errors resulting from interface parameter adjustments or method deprecation. These issues not only increase debugging costs but also may disrupt project iteration. The high-frequency occurrence of such problems in open-source projects directly reflects the urgent need for efficient detection tools. With the popularization of microservices and multi-cloud architectures, systems have become increasingly dependent on third-party APIs; a single project may involve dozens or even hundreds of API versions. Mixed scenarios involving cross-protocols (e.g., REST, gRPC) and multi-data formats (e.g., JSON, PROTOBUF) further exacerbate the difficulty of version management and compatibility verification. Traditional manual detection or mainstream tools (such as Postman and Swagger) focus on functional testing, making it difficult to cover implicit compatibility issues (e.g., semantic-level differences, residual implicit dependencies) and unable to meet detection needs in complex architectures. From the perspective of industry value, API compatibility has become a core element in ensuring system stability—in critical fields such as finance and healthcare, compatibility issues may lead to severe consequences such as transaction interruptions and data errors, resulting in direct economic losses. In cross-enterprise collaboration, version incompatibility prolongs the system integration cycle and reduces the efficiency of ecological collaboration. However, existing technologies lack sufficient capability to analyze API semantic differences and struggle to address detection needs in scenarios involving dynamically generated APIs and multi-role usage. This highlights the research value of detection solutions based on LLMs (Large Language Models) combined with COT (Chain of Thought) and MultiAgent technologies: through logical reasoning to analyze the semantic impact of changes and multi-perspective simulation for collaborative verification, these solutions are expected to break through the limitations of traditional schemes, solve the practical pain points of developers and technical bottlenecks in complex architectures, and hold significant theoretical and practical significance for ensuring system reliability and improving industry collaboration efficiency.

B. API Compatibility Type

C. Motivating Example

III. APPROACH

This section details our methodology for detecting Python API incompatibilities using Large Language Models (LLMs) with structured Chain-of-Thought (COT) prompting. We first outline the overall workflow and core challenges, then elaborate on the design principles compared to existing methods, and technical implementation. This section details our methodology for detecting Python API incompatibilities using Large Language Models (LLMs) with structured Chain-of-Thought (COT) prompting. We first outline the overall workflow and core challenges, then elaborate on the design principles compared to existing methods, and technical implementation.

A. CoT

B. Overview and Challenges

Our approach targets code-level changed API pairs (excluding module-level, class-level, and API addition/deletion changes) between two versions of a Python project. Figure x illustrates the end-to-end workflow. First, we extract all APIs from two versions of the project and filter API pairs to retain only those with code-level modifications. Then we instruct LLM with COT guidance to perform line-by-line code analysis. Finally, The LLM generates a structured JSON output, including detected incompatibility types, detailed reasons for each type, and auxiliary information.

figure x

During this three-step process, we need to address the following three challenges:

- **High-Level Description Misalignment (C1):** Existing LLM-based methods[x] often rely on high-level incompatibility descriptions rather than code-level semantics. This disconnect causes LLMs to struggle with mapping abstract descriptions to concrete code changes.
- **Scalability with Incompatibility Categories (C2):** As the number of incompatibility categories increases, naive prompting leads to exponential growth in prompt token size and LLM runtime. For N categories, this approach requires N independent LLM calls per API pair, making it infeasible for large-scale analysis.
- **Unreliable and Inconsistent Outputs (C3):** LLMs exhibit inherent output randomness—they may produce contradictory judgments for the same prompt (e.g., labeling a parameter rename as “compatible” in one run and “incompatible” in another) or include internally inconsistent reasoning (e.g., citing non-existent code lines to support a claim).
- **Unstructured COT Reasoning (C4):** Without guided thinking frameworks, LLMs’ intermediate reasoning (e.g.,

how they identify a parameter default value change) remains unstructured. This makes it impossible to verify the validity of the LLM’s conclusions.

C. Methodology Design

To address the above challenges, we design a one-pass COT analysis framework with structured prompting, built on insights from prior empirical studies and innovations in LLM guidance.

We first establish a comprehensive taxonomy of Python API incompatibilities, extending the 7-category framework proposed by Shen et al. [x] (a foundational empirical study on Python API evolution). Shen et al. identified incompatibilities rooted in parameters (CI1-CI5), exceptions (CI6), and return values (CI7). Through our own data collection, we validated these 7 categories and discovered two new, previously unreported types:

CI8: global variable change **CI9:** reference variable change
Table x

1) *Limitation of Current Methods:* Traditional static analysis methods (e.g., Shen et al. [2024]) rely on handcrafted rules. However, Rule design requires expert knowledge making it hard to generalize to new libraries or unknown incompatibilities. For example, shen define data structure P to record parameter information for the detection of CI1-CI5. However, without detailed study into incompatible examples. it is hard to design rules for detections. Besides, These methods fail to interpret semantic context while many crucial infomation should be analyzed through variable names, comments, usage in function bodies, etc. For example, Shen et al.’s method cannot recognize that renaming a parameter from arguments to args is a compatibility issue—since the AST pattern for parameter names differs, but the static rule does not link the semantic similarity between arguments and args.

In our method, by leveraging LLMs’ semantic understanding capabilities, our approach directly addresses this limitation. The LLM integrates contextual cues to identify semantic changes, even when syntactic patterns are non-trivial.

2) *One-Pass COT Analysis Framework:* To tackle scalability and output unreliability, we design a one-pass COT framework guided by two core principles: two-stage analysis and unified scanning. First, the LLMs is instructed to process the parameter list, which is syntactically structured, then analyze the function body, which is semantically rich—aligning with how developers naturally review code changes. We embed rules for all 9 incompatibility categories into a single prompt to enable the LLM to detect multiple incompatibilities in one analysis pass, avoiding repeated LLM calls. The framework is divided into four sequential phases, each with predefined data structures and rules to guide the LLM’s reasoning. This design offers three key advantages: efficiency, as runtime is independent of the number of incompatibility categories (all rules are processed in one pass); reduced misjudgment, as it enforces dependencies between interdependent categories (e.g., analyzing CI1 (parameter addition) before CI4 (default value change) to avoid misclassifying added parameters as default value changes); and structured reasoning, as each phase outputs intermediate data structures like parameter mappings

and exception type lists, which make the LLM’s logic traceable and verifiable.

3) *Structured Prompt Design:* The prompt is the backbone of our framework, structured to enforce consistency and guide the LLM’s COT reasoning. It includes four components: Instructions, Analysis Phases, Source Code, and Examples. A simplified template is shown below (full rules are provided in Appendix A):

Analyze two versions of a Python API function (‘code_v1’ and ‘code_v2’) to detect incompatibilities. The output must be in JSON format.

```
### Instructions
Please strictly follow the ‘Analysis Phases’ to identify all incompatibility types (CI1 – CI9) and provide concise reasons. The final output must include:
- ‘additional_info’: Key variables/structures used in analysis (e.g., parameter mappings).
- ‘reasons’: One-sentence explanations linking code changes to incompatibility types.
- ‘incompatibility’: List of detected types

### Analysis Phases
#### Phase 0: Initialization
Initialize the result dictionary:
{
  "incompatibility": [], "reasons": [],
  "additional_info": {},
  "param_mapping": {}, "
  "exception_list_v1": [], ...}
}

#### Phase 1: Parameter List Processing
Extract parameter lists from ‘code_v1’ and ‘code_v2’, then build:

#### Phase 2: Initial Incompatibility Analysis
Use ‘P1’/‘P2’ to update results:

#### Phase 3: Function Body Processing (Line-by-Line)
Initialize data structures for ‘code_v1’/‘code_v2’:

#### Phase 4: Comprehensive Incompatibility Analysis
Use Phase 1–3 data structures to detect remaining types:

### Source Code
"code_v1": ...
"code_v2": ...
```

```
### Examples
}
```

4) *Data Structure and Detection Design*: The analysis leverages several core data structures to capture critical information from function signatures and bodies, facilitating subsequent incompatibility checks. Based on the detecting process for different incompatibility types, we design following data structure which is initialized at begining and update throughout the scanning of the source code.

Parameter Lists (P1, P2): These lists encode the parameters of source code. Each parameter is represented as a tuple (name, position index, type, default value), where type includes positional, optional (with a default value), or kwargs, enabling direct comparison of parameter presence, order, and defaults.

KWARGS: A string storing the name of the kwargs parameter if present in both code v1 and code v2; empty otherwise. This structure anchors the analysis of keyword argument keys accessed within the function body.

ReName: A list of tuples (name1, name2): name1 (from code v1) and name2 (from code v2) are semantically similar or share the same position index, indicating potential parameter renaming.

Keyword Argument Keys (K1, K2): Lists of keys explicitly accessed via kwargs (e.g., `kwargs.get('key')` or `kwargs['key']`) in code v1 and code v2, respectively. These capture dependencies on dynamic keyword arguments.

Renamed Parameter Usage (ReNUsage1, ReNUsage2): Dictionaries mapping parameters in ReName pairs to lists of code lines where they are used. This tracks contextual usage to validate renaming.

Condition Statements (C1, C2): Lists of condition strings (e.g., `if x < 0`) extracted from code v1 and code v2, indexed sequentially to link with dependent logic (e.g., assignments, exceptions).

Data Flow (D1, D2): Lists of tuples (variable name, variable type, dependent variables, condition indices) capturing variable assignments. variable type distinguishes local variable, global variable, and reference variable (e.g., `self`), while dependent variables and condition indices track dependencies and associated conditions.

Exceptions (E1, E2): Lists of tuples pairing explicit exception types (e.g., `ValueError`) with indices of conditions under which they are raised (from C1/C2), excluding implicit behaviors or unthrown exceptions.

Return Values (R1, R2): Lists of tuples pairing return expressions with condition indices, capturing how return values vary under different conditions.

The 9 types of incompatibility can be detected based on the data structure above. The detection rules are showed in Table x.

Table x

D. Multi-Agent-Based Incompatibility Category Validation

To address judgment biases in the incompatibility category candidates generated by the CoT-LLM, we design a multi-agent collaborative validation framework. It implements accurate verification of candidate categories through a process of "test

case generation, simulation execution, and comprehensive evaluation," ultimately filtering out truly valid API incompatibility types. The framework comprises three functionally independent yet collaboratively working agent modules: the Test Case Generation Agent, Simulation Agent, and Comprehensive Analysis Agent. These modules form a validation chain through explicit information interaction, ensuring the objectivity and reliability of validation results.

The Test Case Generation Agent serves as the initiator of the validation process, tasked with constructing targeted test inputs to probe potential incompatibilities. Its core functionality is to generate pairs of calling statements—one for the old API version and one for the new—along with supplementary context, based on the provided old and new API code snippets, the candidate incompatibility category, and the underlying reasoning. The design of these test cases adheres to a critical principle: they must be highly likely to trigger the alleged incompatibility if it indeed exists. For instance, when evaluating a candidate issue related to changed default parameter values, the agent will craft calling statements that omit the affected parameters, thereby exposing discrepancies in behavior caused by the default value shift. The output of this agent follows a structured format, explicitly separating the calling statements and supplementary information for the old and new APIs to ensure clarity in subsequent processing.

Upon receiving the test cases from the Test Case Generation Agent, the Simulation Agent takes on the role of executing these tests in a controlled environment. Its primary responsibility is to simulate the runtime behavior of both the old and new API versions when invoked with the generated test inputs. This involves parsing the API code, instantiating necessary objects (if applicable), and executing the calling statements step-by-step. The agent meticulously records the execution results: if the program exits normally with a return value, it captures the specific output; if an exception is raised (e.g., missing arguments, type errors), it documents the error type and description. By faithfully replicating runtime scenarios, this agent transforms abstract code differences into concrete behavioral outcomes, providing empirical data for validating the alleged incompatibility.

The Comprehensive Analysis Agent acts as the final evaluator, synthesizing information from the preceding stages to assess the validity of the candidate incompatibility category. It takes as input the definition of the incompatibility type, the reasoning behind the initial diagnosis, and the behavioral results generated by the Simulation Agent. Using a scoring mechanism ranging from 0 to 10, the agent evaluates the alignment between the diagnosed incompatibility, the test case design, and the observed runtime behaviors. The score is determined based on three key criteria: logical consistency (whether the reasoning logically leads to the alleged incompatibility), rigor of evidence (whether the simulation results clearly demonstrate the expected behavioral discrepancy), and coherence (whether all components of the validation process reinforce the same conclusion). A higher score indicates greater confidence in the existence of the specific incompatibility, while a lower score suggests either flaws in the initial diagnosis or inadequacies in the test scenario.

The collaborative workflow of the three agents forms a closed-loop validation pipeline. Starting with a candidate incompatibility category, the Test Case Generation Agent tailors probes to expose potential issues, the Simulation Agent generates empirical behavioral data, and the Comprehensive Analysis Agent renders a data-driven judgment. This division of labor ensures that each stage of validation is handled by a specialized component, reducing the risk of subjective biases that might arise from a single model’s judgment. By structuring the validation as an explicit information flow—from test design to execution to analysis—the framework enhances the transparency and reproducibility of the verification process. Ultimately, this multi-agent approach strengthens the reliability of incompatibility category identification, providing a robust mechanism to filter out spurious candidates and retain only those that are empirically validated.

E. implementation

We implement the approach using Python, leveraging open-source tools and APIs to balance reproducibility and efficiency. For API extraction, we combine the GitHub API and Python’s standard ast library: the GitHub API fetches source code for specific project versions to enable automated data collection, while the ast library builds ASTs for function-level slicing—extracting only the function body (excluding imports or global statements outside the API). For LLM interaction, we use DeepSeek-chat API with temperature 0 to minimize randomness and max-tokens 2048 to accommodate structured outputs.

IV. EVALUATION

A. Dataset Construction

The Python third-party library Application Programming Interface (API) compatibility issue dataset constructed in this study comprises two core components: a benchmark dataset derived from existing research, which includes 108 incompatible API pairs built by Shen et al. on the Flask and pandas libraries; and an extended dataset supplementary constructed in this work. Through systematic repository screening, Issue/Pull Request (PR) analysis, and multi-round validation, the extended dataset adds over 70 valid samples, resulting in a total of more than 130 compatibility issue annotations. The dataset comprehensively covers third-party libraries across multiple application domains, such as web development, data analysis, and cloud services, and includes API compatibility issues across major, minor, and patch version transitions. This design provides multi-scenario and fine-grained experimental data support for research on Python third-party library API compatibility detection.

Dataset A.

To construct the dataset of 108 incompatible API pairs, Shen et al. selected six version pairs from two representative Python third-party libraries (Flask for web development and pandas for data analysis), covering cross-major, cross-minor, and cross-patch version transitions. A combined approach of regression testing and version update log analysis was adopted to ensure the comprehensiveness and reliability of the dataset. For regression testing, test cases from the previous

TABLE I: Compatibility Incompatibility Categories (CI) — detection & rules

CI	Name	Data-structure construction rules	Incompatibility identification rules
CI1	Parameter Addition / Deletion	Signature-level: parameter appears or disappears between versions. Build P1 and P2: list parameters as tuples (name, position index, type, default).	Trigger when: a parameter exists in P1 but not in P2; or a positional parameter exists in P2 but not in P1; or an optional parameter exists in P2 and causes positional incompatibility.
CI2	Kwargs Key Addition / Deletion	If both versions have ‘**kwargs’, set ‘KWARGS’ = param name; extract K1 and K2: explicit keys accessed via ‘kwargs.get’ or ‘kwargs[‘key’]’. Statements to detect include body expressions like ‘kwargs.get(‘key’)’ or ‘kwargs[‘key’]’.	Trigger when ‘KWARGS’ is non-empty and K1 $\neq$ K2 (added or removed).
CI3	Parameter Re-name	Produce ReName pairs (name1, name2) by semantic or positional match; collect ReNUsage1/2: lines where each renamed parameter is used; collect D1/D2 for data-flow context. Detect from signature name pairs and body lines that use the parameter (assignments/usages).	Trigger when a ReName pair exists and ReNUsage pairs are highly similar (identical or structurally identical) or flow where only variable names differ.
CI4	Default Value Change	Compare P1 and P2 default value fields for same-named parameters (or matched by position if appropriate). Statements to detect: function signature default values.	Trigger when a parameter exists in both P1 and P2 but their default values differ.
CI5	Parameter Range / Condition Change (affects exceptions)	Build C1/C2 (condition strings), E1/E2 (explicit raises with condition index), and D1/D2 to map conditions to parameters. Statements to detect include condition statements (‘if’, ‘while’, ‘try’ scopes) and ‘raise’ / ‘assert’ lines referencing parameters.	Trigger when exception conditions tied to a parameter are present in both versions so that raising behavior checks differ.
CI6	Exception Added/Delete or Type Change	Record E1/E2: explicit exception types with associated condition indices. Statements to detect: ‘raise ExceptionType(...)’ and ‘assert’ lines in function body.	Trigger when exception types differ between E1 and E2 (added, removed, or changed). If $\text{len}(E1) \neq \text{len}(E2)$, (and often CI5 as well).
CI7	Return Value Change	Collect R1/R2: return expressions with condition indices; use D1/D2 to trace data-flow dependencies of returned variables. Statements to detect: ‘return’ (or ‘yield’) statements and their returned expressions.	Trigger when a return expression exists in one version but not in the other or when the return flow producing the result differs in a way that affects content/type/structure.
CI8	Global Variable Change	In D1/D2, mark entries with variable type = ‘global variable’ and record dependent variables and conditions. Statements to detect: assignment statements referencing names classified as global; ‘global’ / ‘nonlocal’ markers.	Trigger when global variable data-flow differs between versions and those differences change the global variable content or visible state.
CI9	Reference Parameter Change (e.g., ‘self’)	In D1/D2, mark entries with variable type = ‘reference variable’ (including ‘self’); collect attribute assignment sites and dependency lists. Statements to detect: assignment statements to reference variables or their attributes, direct reassignments of reference parameter.	Trigger when: the reference parameter itself is reassigned; a new attribute is assigned to the parameter in one version but not the other; or an existing attribute is reassigned, has materially different logic/dependencies.

version (v1) of each library were executed on the updated version (v2); failed test cases indicated potential incompatible APIs, leading to the identification of 70 incompatible API pairs from 58 failed test files among 2,290 regression test files across the six version pairs. Complementarily, version update logs (including intermediate logs for cross-patch version pairs to capture persistent changes) were analyzed to identify additional incompatible APIs by targeting keywords such as "parameter reduction", "added processing", and "no longer supported", resulting in 38 supplementary incompatible API pairs from 11 analyzed logs. To enhance data credibility, the collection process was independently conducted by the first two authors, with discrepancies resolved through group discussions organized by the third author, involving joint reviews of source code, regression test results, and update logs to confirm the existence of compatibility issues for controversial API pairs. This dual-method strategy effectively mitigated limitations associated with incomplete test coverage in unit testing and inconsistent log quality, ultimately forming a dataset of 108 incompatible API pairs that served as the foundation for subsequent empirical analysis.

Dataset B.

We adopted a different strategy to construct our own dataset to cover codes from more areas. We first selected the top 10 Python libraries by download volume over the past 30 days (as of May 1, 2025) from "https://hugovk.github.io/top-pypi-packages/top-pypi-packages-30-days.json". Those libraries are the most popular ones and can represent most python code usage. However, to cover most compatibility issues, for each candidate repository, we also selected additional relevant Python repositories because some issues are not directly reported in the source repository but in relevant Python repositories.

In the preliminary screening of Issues and PRs, for each selected repository, we cycled through a list of keywords (including 'compatibility', 'compatible', 'version', 'evolution', 'exception', 'TypeError', 'AttributeError', 'ImportError', 'breaking changes') and used the GitHub Issues Search API (/search/issues) to retrieve issues and PRs matching these keywords.

Thereafter, approximately 400 entries were filtered through two rounds of LLM-based checks, followed by manual review. In the LLM analysis phase, the LLM was employed to extract structured information from the issues, including username, repository, version tag1, version tag2, file path, object name, and llm analyse compatibility. Multiple GitHub checks were conducted on the extracted results to ensure the validity of the fields. Specifically, the first LLM was used to convert raw issues into structured compatibility problem entries, which constituted the core information extraction of the dataset, extracting key information such as error type, root cause, library, version, affected files, breaking change, and recommended version. Additionally, the second LLMs was used to supplement and explain the technical details of compatibility issues, especially the causal chain between API changes and errors, serving as an in-depth evidence supplement of the dataset.

After filtering of LLMs, from the original 2000 samples, xxx was ...

Finally, manual verification was performed to confirm the existence of incompatibilities and the correctness of Python API paths, followed by annotation. Ultimately, over 70 samples were obtained, with a total of more than 130 annotations added.

TABLE II: Top Python Libraries by Download Volume

oprule Library	Category	relevant project counts
boto3	Web development	9
urllib3	Web development	7
requests	Web development	10
botocore	Web development	6
certifi	Security	3
charset	xxx	0
normalizer	xxx	0
idna	xxx	3
setuptools	xxx	10
aiobotocore	xxx	3
python-dateutil	xxx	2

B. Experiment Setup

- **Accuracy Evaluation (RQ1):** How is the overall accuracy of TOOL compared with the state-of-the-arts?
- **Effectiveness Evaluation (RQ2):** How is the effectiveness of TOOL in different API compatibility types?
- **LLM Generalizability (RQ3):** How is the accuracy of TOOL using different LLMs?
- **Ablation Study (RQ4):**

Baselines. We selected XXX , XXX, and XXX as the comparison.

Metrics. We use precision and recall as metrics, which are defined as follows:

C. Results

- 1) *Accuracy Evaluation (RQ1):*
- 2) *RQ2:*
- 3) *RQ3:*
- 4) *RQ4:*

V. RELATED WORK

A. API Compatibility Detection

The continuous evolution of software libraries inevitably introduces breaking changes, posing severe threats to the stability of downstream applications. In a systematic review, Lamothe et al.[15] identified that understanding the specific impacts of API evolution on developers and users remains a paramount challenge. Substantiating this concern, extensive empirical studies have been conducted to characterize the pervasiveness of compatibility issues across diverse ecosystems. In the mobile domain, Cai et al.[3] conducted a large-scale longitudinal study on Android runtime compatibility, while Liu et al.[20] summarized five categories of API-induced issues. More recently, the focus has deepened into device-specific behaviors [7] and compatibility issues caused by vendor customizations [4]. Beyond mobile, such challenges are widespread in system software like Linux [30], compiled languages like Java—where behavioral breaking changes frequently affect client applications [11]—and the NPM ecosystem [13]. Even in complex deep learning systems,

compatibility issues spanning software and hardware stacks have been rigorously characterized [33]. However, distinct from these ecosystems, Python presents unique challenges. Zhang et al.[41] revealed that Python framework APIs evolve with patterns significantly different from Java. Recent works have further classified Python-specific incompatibility symptoms[44] and cross-repository conflicts [38], underscoring that the dynamic nature of Python makes the compatibility issue more complicated.

A mainstream of the related work apply dynamic techniques. The core strategy of these approaches involves constructing and executing test cases to trigger API behaviors across different versions. For instance, Sun et al.[28] proposed JUnitTestGen, which mines real-world API usages to generate executable unit tests. Going a step further to ensure comprehensive coverage, Mahmud et al.[24] introduced APICIA, which performs a bidirectional impact analysis. Besides, to address client-specific upgrades, Zhu et al.[43] utilized a knowledge-guided framework to generate incompatibility-revealing tests, while Chen et al.[5] leveraged cross-project testing to detect behavioral backward incompatibilities in Python. Beyond standard testing, more sophisticated dynamic techniques have been introduced to analyze complex dependency chains. Kong et al.[14] employed dynamic object relation graphs to precisely detect breaking changes in JavaScript. Furthermore, effective dynamic analysis relies on valid execution environments, a challenge addressed by tools like ReadPyE[6] which automates the generation of compatible Python runtimes.

However, the applicability of dynamic analysis is often constrained by its high computational costs and reliance on existing test suites, which can leave parts of the application untested. In contrast, static analysis approaches offer the advantage of analyzing the entire codebase without execution, providing a scalable alternative. In the Android ecosystem, early research progressed along two parallel dimensions: device fragmentation and general API evolution. Addressing fragmentation, Wei et al.[35] pioneered FicFinder to characterize device-specific issues, a direction later advanced by Pivot[36] to learn API-device correlations from large corpora. In parallel, to tackle general API evolution, Li et al.[18] introduced CiD to model API lifecycles, while He et al.[9] developed IctApiFinder to detect incompatible usages. As the field matured, techniques evolved towards automated repair and finer granularity. Xia et al.[37] designed RAPID to mine repair templates from application history. Concurrently, detection depth increased significantly: from general methods to complex callbacks (ACID[25], Huang et al.[10]) and field-level evolutions (FCID[26], AndroMevol [21]), with recent work identifying recurring design smells [12]. Beyond Android, static techniques have advanced in other domains. In Java, tools like APIDiff [2] and the recent source-based Roseau [16] focus on identifying breaking changes. In the Python domain, static analysis has been tailored to its specific ecosystem. Zhang et al.[41] developed PyCompat to handle framework API evolution, while Zhao et al.[42] constructed knowledge bases to detect version incompatibilities in deep learning stacks. Other tools target specific risks such as breaking changes in packages (Aexpy [8]), deprecated APIs (APIScanner [31]),

and complex variadic parameters [39]. More recently, static approaches have expanded to classify Python incompatibility types [44] and solve cross-repository issues via offline database construction [38].

Despite the advancements in static analysis, these approaches encounter severe bottlenecks when applied to dynamic languages. Python’s dynamic features, such as runtime type inference and dynamic attribute access, often evade capture by rigid control-flow or data-flow analysis, resulting in high false positive rates or missed subtle semantic changes. To bridge these gaps, our work integrate LLMs, enhanced via Chain-of-Thought (CoT) reasoning and multi-agent collaboration. This approach capitalizes on LLMs’ ability to interpret implicit API behaviors, infer context-dependent compatibility rules, and adapt to diverse usage scenarios—capabilities that resolve the semantic detection limitations of all preceding traditional static methods.

B. LLMs for Library Evolution

With the rapid advancement of Large Language Models (LLMs), researchers have actively explored their potential in automating complex tasks within the domain of library evolution, leveraging their semantic understanding.

Researchers have applied LLMs to address various challenges in library evolution, primarily focusing on two distinct tasks: managing deprecated APIs and adapting code generation. For Deprecated API management, Mahmud et al.[27] proposed GUPPY, the first LLM-based approach to automatically update Android deprecated API usages without requiring manual templates. Similarly, to prevent the introduction of outdated APIs during generation, Bai et al.[1] developed APILOT, which leverages real-time differential analysis to verify LLM outputs against a database of outdated APIs. Regarding API Usage Adaptation and robust code generation, Li et al. [17] introduced Structured Chain-of-Thought (SCoT). By incorporating programming structures (sequence, branch, loop) into intermediate reasoning steps, SCoT aims to solve the low accuracy of LLMs in generating complex, structurally correct code that adapts to specific requirements.

To enhance the performance of LLMs in various tasks, advanced prompting and architectural techniques have been employed. Chain-of-Thought (CoT), originally proposed by Wei et al.[34] to elicit reasoning, has evolved significantly. Zhang et al.[40] enhanced CoT by constructing task-specific reasoning patterns. In the specific domain of code synthesis, Li et al. [17] further advanced this by proposing Structured CoT (SCoT), which explicitly embeds programming logic (e.g., branches and loops) into the reasoning chain to constrain the model’s output. Meanwhile, Liu et al.[22] proposed ERA-CoT to capture entity relationships for better context understanding. Agent-based system is another outstanding architectural technique. Ma et al.[23] proposed iAudit, a multi-agent framework combining a Ranker and Critic to debate vulnerabilities in smart contracts. Similarly, Sun et al.[29] developed GPTScan, which combines GPT’s capability in recognizing key variables with static program analysis to detect logic vulnerabilities.

Despite these advancements, fundamental challenges persist in applying LLMs to library evolution. Wang et al.[32] quanti-

tatively revealed that LLMs frequently utilize deprecated APIs due to outdated training data. Liang et al.[19] further confirmed that the knowledge cutoff significantly hampers performance, although Retrieval-Augmented Generation (RAG) can mitigate this. Critically, existing solutions leave a methodological gap: current CoT approaches and multi-agent frameworks lack design for API compatibility issues. To address these limitations, our work introduces a specialized multi-agent framework. We design a CoT prompt mechanism to explicitly reason about incompatibility categories and orchestrate Input-Generator and Simulator agents to construct a closed-loop verification system.

REFERENCES

- [1] W. Bai, K. Xuan, P. Huang, Q. Wu, J. Wen, J. Wu, and K. Lu, “Apilot: Navigating large language models to generate secure code by sidestepping outdated api pitfalls,” *arXiv preprint arXiv:2409.16526*, 2024.
- [2] A. Brito, L. Xavier, A. Hora, and M. T. Valente, “Apidiff: Detecting api breaking changes,” in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2018, pp. 507–511.
- [3] H. Cai, Z. Zhang, L. Li, and X. Fu, “A large-scale study of application incompatibilities in android,” in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 216–227.
- [4] J. Chen, K. Li, Y. Chen, L. Wei, and Y. Liu, “Demystifying device-specific compatibility issues in android apps,” in *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2024, pp. 525–537.
- [5] L. Chen, F. Hassan, X. Wang, and L. Zhang, “Taming behavioral backward incompatibilities via cross-project testing and analysis,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 112–124.
- [6] W. Cheng, W. Hu, and X. Ma, “Revisiting knowledge-based inference of python runtime environments: A realistic and adaptive approach,” *IEEE Transactions on Software Engineering*, vol. 50, no. 2, pp. 258–279, 2023.
- [7] Z. Dong, Y. Zhao, T. Liu, C. Wang, G. Xu, G. Xu, L. Zhang, and H. Wang, “Same app, different behaviors: Uncovering device-specific behaviors in android apps,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2099–2109.
- [8] X. Du and J. Ma, “Aexpy: Detecting api breaking changes in python packages,” in *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2022, pp. 470–481.
- [9] D. He, L. Li, L. Wang, H. Zheng, G. Li, and J. Xue, “Understanding and detecting evolution-induced compatibility issues in android apps,” in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 167–177.
- [10] H. Huang, L. Wei, Y. Liu, and S.-C. Cheung, “Understanding and detecting callback compatibility issues for android applications,” in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 532–542.
- [11] D. Jayasuriya, V. Terragni, J. Dietrich, and K. Blincoe, “Understanding the impact of apis behavioral breaking changes on client applications,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1238–1261, 2024.
- [12] W. Jin, J. Shang, J. Zheng, M. Sun, Z. Huang, M. Fan, and T. Liu, “The design smells breaking the boundary between android variants and aosp,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2025, pp. 599–599.
- [13] D. Kong, J. Liu, L. Bao, and D. Lo, “Toward better comprehension of breaking changes in the npm ecosystem,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, pp. 1–23, 2025.
- [14] D. Kong, J. Liu, C. Ni, D. Lo, and L. Bao, “More effective javascript breaking change detection via dynamic object relation graph,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSA, pp. 2340–2361, 2025.
- [15] M. Lamothe, Y.-G. Guéhenec, and W. Shang, “A systematic review of api evolution literature,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 8, pp. 1–36, 2021.
- [16] C. Latappy, T. Degueule, J.-R. Falleri, R. Robbes, and L. Ochoa, “Roseau: Fast, accurate, source-based api breaking change analysis in java,” *arXiv preprint arXiv:2507.17369*, 2025.
- [17] J. Li, G. Li, Y. Li, and Z. Jin, “Structured chain-of-thought prompting for code generation,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–23, 2025.
- [18] L. Li, T. F. Bissyandé, H. Wang, and J. Klein, “Cid: Automating the detection of api-related compatibility issues in android apps,” in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2018, pp. 153–163.
- [19] L. Liang, J. Gong, M. Liu, C. Wang, G. Ou, Y. Wang, X. Peng, and Z. Zheng, “Rustevor 2: An evolving benchmark for api evolution in llm-based rust code generation,” *arXiv preprint arXiv:2503.16922*, 2025.
- [20] P. Liu, Y. Zhao, H. Cai, M. Fazzini, J. Grundy, and L. Li, “Automatically detecting api-induced compatibility issues in android apps: a comparative analysis (replicability study),” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 617–628.
- [21] P. Liu, Y. Zhao, M. Fazzini, H. Cai, J. Grundy, and L. Li, “Automatically detecting incompatible android apis,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 1, pp. 1–33, 2023.
- [22] Y. Liu, X. Peng, T. Du, J. Yin, W. Liu, and X. Zhang, “Era-cot: improving chain-of-thought through entity relationship analysis,” *arXiv preprint arXiv:2403.06932*, 2024.
- [23] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, “Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications,” *arXiv preprint arXiv:2403.16073*, 2024.
- [24] T. Mahmud, M. Che, J. Rouijel, M. Khan, and G. Yang, “Apicia: An api change impact analyzer for android apps,” in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 99–103.
- [25] T. Mahmud, M. Che, and G. Yang, “Acid: an api compatibility issue detector for android apps,” in *Proceedings of the ACM/IEEE 44th international conference on software engineering: Companion proceedings*, 2022, pp. 1–5.
- [26] T. Mahmud, M. Che, and G. Yang, “An empirical study on compatibility issues in android api field evolution,” *Information and Software Technology*, vol. 175, p. 107530, 2024.
- [27] T. Mahmud, B. Duan, M. Che, A. Yasmin, A. H. Ngu, and G. Yang, “Automated update of android deprecated api usages with large language models,” *arXiv preprint arXiv:2411.04387*, 2024.
- [28] X. Sun, X. Chen, Y. Zhao, P. Liu, J. Grundy, and L. Li, “Mining android api usage to generate unit test cases for pinpointing compatibility issues,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.
- [29] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, “Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [30] C.-C. Tsai, B. Jain, N. A. Abdul, and D. E. Porter, “A study of modern linux api usage and compatibility: What to support when you’re supporting,” in *Proceedings of the Eleventh European Conference on Computer Systems*, 2016, pp. 1–16.
- [31] A. Vadlamani, R. Kalicheti, and S. Chimalakonda, “Apiscanner-towards automated detection of deprecated apis in python libraries,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2021, pp. 5–8.
- [32] C. Wang, K. Huang, J. Zhang, Y. Feng, L. Zhang, Y. Liu, and X. Peng, “Llms meet library evolution: Evaluating deprecated api usage in llm-based code completion,” *arXiv preprint arXiv:2406.09834*, 2024.
- [33] J. Wang, G. Xiao, S. Zhang, H. Lei, Y. Liu, and Y. Sui, “Compatibility issues in deep learning systems: Problems and opportunities,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 476–488.
- [34] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou et al., “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [35] L. Wei, Y. Liu, and S.-C. Cheung, “Taming android fragmentation: Characterizing and detecting compatibility issues for android apps,” in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 226–237.
- [36] L. Wei, Y. Liu, and S.-C. Cheung, “Pivot: learning api-device correlations to facilitate android compatibility issue detection,” in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 878–888.
- [37] H. Xia, Y. Zhang, Y. Zhou, X. Chen, Y. Wang, X. Zhang, S. Cui, G. Hong, X. Zhang, M. Yang et al., “How android developers handle evolution-induced api compatibility issues: A large-scale study,” in *Proceedings of*

- the *ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 886–898.
- [38] Y. Xie, Z. Jia, S. Li, Y. Wang, J. Ma, X. Li, H. Liu, Y. Fu, and X. Liao, “How to pet a two-headed snake? solving cross-repository compatibility issues with hera,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 694–705.
 - [39] S. Zhang, G. He, and G. Xiao, “Coding pitfalls: Demystifying the potential api compatibility risk of variadic parameters in python,” in *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2024, pp. 105–106.
 - [40] Y. Zhang, X. Wang, L. Wu, and J. Wang, “Enhancing chain of thought prompting in large language models via reasoning patterns,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25 985–25 993.
 - [41] Z. Zhang, H. Zhu, M. Wen, Y. Tao, Y. Liu, and Y. Xiong, “How do python framework apis evolve? an exploratory study,” in *2020 IEEE 27th international conference on software analysis, evolution and reengineering (saner)*. IEEE, 2020, pp. 81–92.
 - [42] Z. Zhao, B. Kou, M. Y. Ibrahim, M. Chen, and T. Zhang, “Knowledge-based version incompatibility detection for deep learning,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 708–719.
 - [43] C. Zhu, M. Zhang, X. Wu, X. Xu, and Y. Li, “Client-specific upgrade compatibility checking via knowledge-guided discovery,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–31, 2023.
 - [44] , , , and , “python api ,” , vol. 36, no. 4, pp. 1435–1460, 2024.